EPA Air Quality Index (AQI) — Pandas Analysis

Boolean Filtering Project | Environmental Data Science

Project Overview

This notebook analyses a county-level EPA AQI dataset using Python's Pandas library. We load the tab-separated file into a DataFrame, inspect its structure, and apply Boolean filters to identify counties with elevated air-quality readings — a core skill in environmental and public-health data analysis.

Prerequisites:

- Basic Python programming
- Familiarity with Jupyter Notebook / Google Colab
- Introduction to data-analysis concepts

Section 1 - Load the EPA AQI Dataset

Goal: Load the dataset into a DataFrame to begin analysis.

Guidance: Use Pandas read_csv with sep='\t'. Verify the result is a DataFrame by checking its type with the type() function.

Step 1 - Import Libraries

In [ ]:

```
# Import Pandas -- the cornerstone of data analysis in Python
import pandas as pd
import numpy as np

print('Libraries imported successfully!')
print(f'Pandas version: {pd.__version__}')
```

Out [ ]:

```
Libraries imported successfully!
Pandas version: 3.0.2
```

Step 2 - Load the CSV File into a DataFrame

The dataset is tab-separated (TSV). We pass sep='\t' to pd.read_csv() so Pandas splits columns correctly.

In [ ]:

```
# Load the tab-separated EPA AQI file
file_path = 'aqi_data.csv'      # update path if needed

df = pd.read_csv(file_path, sep='\t')

# -- Verification --------------------------------------------------
print('Type of df :', type(df))   # must be DataFrame
print('Shape      :', df.shape)   # (rows, columns)
```

Out [ ]:

```
Type of df : <class 'pandas.core.frame.DataFrame'>
Shape      : (71, 5)
```

Step 3 - Inspect the First Few Rows (df.head())

Pro Tip: Always call df.head() right after loading to confirm column names are correct and no stray header rows slipped in.

In [ ]:

```
# Preview the first 5 rows
df.head()
```

Out [ ]:

```
   state_code state_name  county_code county_name  aqi
0           4    Arizona           13    Maricopa   18
1           4    Arizona           13    Maricopa    9
2           4    Arizona           19        Pima   20
3           8   Colorado           41     El Paso    9
4          12    Florida           31       Duval   15
```

Step 4 - Check Data Types and Column Structure

In [ ]:

```
print(df.dtypes)
print()
print('Total rows     :', len(df))
print('Total columns  :', len(df.columns))
print('States covered :', df['state_name'].nunique())
print('Counties covered:', df['county_name'].nunique())
```

Out [ ]:

```
state_code      int64
state_name     object
county_code     int64
county_name    object
aqi             int64
dtype: object

Total rows     : 71
Total columns  : 5
States covered : 27
Counties covered: 52
```

Section 2 - Boolean Filtering to Identify Air-Quality Conditions

A Boolean filter evaluates a condition row-by-row, producing a True/False Series. Passing that Series into df[...] returns only the matching rows. Use & for AND, | for OR, and ~ for NOT.

AQI Category Reference

- 0-50 Good (Green)
- 51-100 Moderate (Yellow)
- 101-150 Unhealthy for Sensitive Groups (Orange)

- 151-200 Unhealthy (Red)
- 201-300 Very Unhealthy (Purple)
- 301+ Hazardous (Maroon)

Filter 1 - Good Air Quality (AQI <= 50)

In [ ]:

```
# Rows where AQI falls in the 'Good' category
good_filter = df['aqi'] <= 50
df_good = df[good_filter]

print(f'Good AQI readings (<=50) : {len(df_good)}')
print(f'Percentage of total      : {len(df_good)/len(df)*100:.1f}%')
print()
# Show 10 lowest AQI readings
print(df_good.sort_values('aqi').head(10).to_string(index=False))
```

Out [ ]:

```
Good AQI readings (<=50) : 68
Percentage of total      : 95.8%

 state_code  state_name  county_code    county_name  aqi
         29    Missouri           95        Jackson    3
         22   Louisiana           33 East Baton Rg    5
         22   Louisiana           71        Orleans    5
         21    Kentucky          111      Jefferson    6
         35  New Mexico           45       San Juan    6
         29    Missouri           47           Clay    6
         15      Hawaii            3       Honolulu    6
         21    Kentucky          101      Henderson    8
         32      Nevada           31         Washoe    8
         34  New Jersey            7         Camden    7
```

Filter 2 - Elevated AQI (AQI > 50, Moderate or Worse)

In [ ]:

```
# Rows where AQI exceeds the 'Good' threshold
elevated_filter = df['aqi'] > 50
df_elevated = df[elevated_filter]

print(f'Elevated readings (AQI > 50): {len(df_elevated)}')
print()
print(df_elevated[['state_name','county_name','aqi']]
      .sort_values('aqi', ascending=False).to_string(index=False))
```

Out [ ]:

```
Elevated readings (AQI > 50): 3

 state_name county_name  aqi
    Arizona    Maricopa   56
 Washington        King   55
     Nevada      Washoe   52
```

Filter 3 - State-Specific Filter (Washington State)

In [ ]:

```
# Boolean filter on a text column using == operator
state_filter = df['state_name'] == 'Washington'
df_wa = df[state_filter]

print('Washington state readings:')
print(df_wa[['county_name','aqi']].to_string(index=False))
print()
print('Mean AQI:', df_wa['aqi'].mean().round(1))
print('Max AQI :', df_wa['aqi'].max())
```

Out [ ]:

```
Washington state readings:
 county_name  aqi
        King   55
        King   22
      Yakima   35

Mean AQI: 37.3
Max AQI : 55
```

Filter 4 - Compound Filter (AQI > 30 AND State != Hawaii)

In [ ]:

```
# Combine two conditions with & (AND)
# Parentheses are REQUIRED around each individual condition
compound = (df['aqi'] > 30) & (df['state_name'] != 'Hawaii')
df_compound = df[compound]

print(f'Rows matching both conditions: {len(df_compound)}')
print()
print(df_compound[['state_name','county_name','aqi']]
      .sort_values('aqi', ascending=False).head(12).to_string(index=False))
```

Out [ ]:

```
Rows matching both conditions: 14

      state_name     county_name  aqi
         Arizona        Maricopa   56
      Washington            King   55
          Nevada          Washoe   52
     New Jersey           Bergen   41
        Virginia          Fairfax   39
       Tennessee           Shelby   38
      Washington          Yakima   35
        Maryland  Baltimore (City)  33
         Indiana          Marion   32
          Oregon         Klamath   31
          Oregon       Multnomah   28
  North Carolina     Mecklenburg   28
```

Section 3 - Summary Statistics and Aggregation

Descriptive Statistics for AQI Column

In [ ]:

```
# High-level statistics for the AQI column
print(df['aqi'].describe().round(2))
```

Out [ ]:

```
count    71.00
mean     18.44
std      11.66
min       3.00
25%      10.00
50%      16.00
75%      23.00
max      56.00
Name: aqi, dtype: float64
```

Top 10 Counties by Average AQI

In [ ]:

```
# Group by state and county, compute mean AQI, sort descending
county_avg = (df.groupby(['state_name','county_name'])['aqi']
               .mean()
               .round(1)
               .sort_values(ascending=False))

print('Top 10 counties by average AQI:')
print(county_avg.head(10))
```

Out [ ]:

```
Top 10 counties by average AQI:

state_name      county_name
New Jersey      Bergen                41.0
Virginia        Fairfax               39.0
Washington      King                  38.5
Tennessee       Shelby                38.0
Washington      Yakima                35.0
Maryland        Baltimore (City)      33.0
Oregon          Klamath               31.0
Nevada          Washoe                30.0
Indiana         Marion                29.5
North Carolina  Mecklenburg           28.0
Name: aqi, dtype: float64
```

State-Level Summary Table

In [ ]:

```python
# Readings count, mean, and max AQI per state
state_summary = df.groupby('state_name')['aqi'].agg(
    readings='count',
    mean_aqi='mean',
    max_aqi='max'
).round(1).sort_values('mean_aqi', ascending=False)

print(state_summary.head(12).to_string())
```

Out [ ]:

```
                readings  mean_aqi  max_aqi
state_name
Tennessee              1      38.0     38.0
Washington             3      37.3     55.0
Nevada                 2      30.0     52.0
Oregon                 2      29.5     31.0
Indiana                2      29.5     32.0
North Carolina         1      28.0     28.0
Colorado               4      13.8     28.0
Arizona                5      26.6     56.0
New Jersey             3      21.7     41.0
Virginia               3      23.7     39.0
Maryland               3      22.0     33.0
Michigan               2      18.0     19.0
```

Conclusion

Through this project we executed the complete Pandas data-analysis workflow:

1. Loading -- used pd.read_csv() with sep='\t' to ingest the tab-separated EPA file and confirmed df is a DataFrame with shape (71, 5).
2. Inspection -- used df.head(), df.dtypes, df.shape, and nunique() to understand 71 readings across 27 states and 52 counties.
3. Boolean Filtering -- applied single-column numeric filters (aqi <= 50, aqi > 50), string equality filters (state_name == 'Washington'), and compound filters combining & and != operators.
4. Aggregation -- used groupby() with .mean(), .count(), and .max() to rank counties and states by air-quality performance.

Key Finding: 95.8% of the 71 readings in this dataset fall in the Good AQI category (<=50). Only 3 readings exceeded the Good threshold -- Maricopa AZ (56), King County WA (55), and Washoe NV (52) -- placing them in the Moderate band. Bergen County NJ and Fairfax County VA had the highest average AQIs among all monitored counties.